

Model-Based Collaborative Filtering

Latent Factor Models

Alberto Paccanaro

EMap – FGV

<https://paccanarolab.org>

O material e as imagens nestes slides são de (ou adaptadas de):

Charu C. Aggarwal, *Recommender Systems – The Textbook*, Springer, 2016 – Chapter 3

D. Lee, H. S. Seung *Learning the parts of objects by non-negative matrix factorization*, Nature, 1999

Neighbourhood-based vs Model-based

- Collaborative filtering: data about past user activities/ preferences (**ratings**)

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U ₁	1			5		2
U ₂		5			4	
U ₃	5	3		1		
U ₄			3			4
U ₅				3	5	
U ₆	5		4			

(a) Ordered ratings

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U ₁	1			1		1
U ₂		1			1	
U ₃	1	1		1		
U ₄			1			1
U ₅				1	1	
U ₆	1		1			

(b) Unary ratings

- Neighbourhood methods are instance-based methods:** a model is not specifically created up front for prediction.
- Model-based methods: create a model of the data.** The training is clearly separated from the prediction.

Different ML models can be generalized to the collaborative filtering scenario

Latent Factor Models

FACTS:

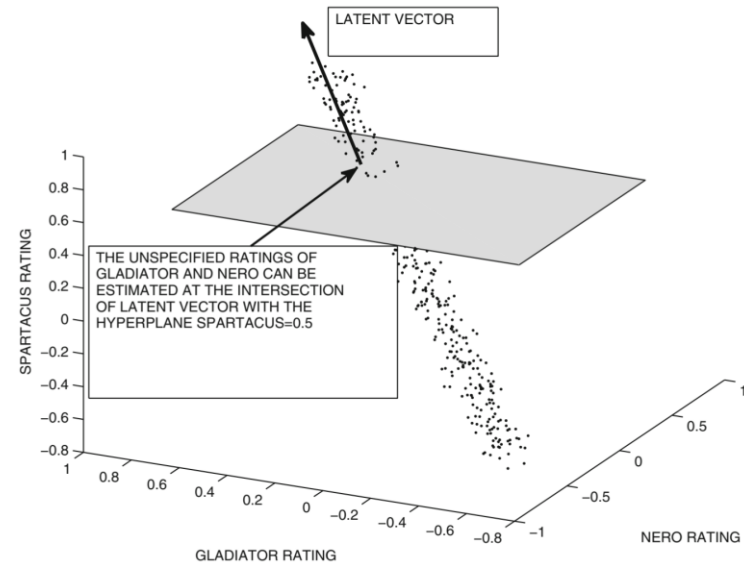
- similar users display similar patterns of rating behaviour
- similar items receive similar ratings
- The rows and columns of data matrices are highly correlated.
- The data matrix is often approximated quite well by a low-rank matrix.

Because of the inherent redundancies in the data, the fully specified low rank approximation can be determined even with a small subset of the entries in the original matrix.

Latent factor models are considered state-of-the-art in recommender systems when interpretability, scalability, and ease of deployment matter.

Intuition – geometric

- Ratings matrix, continuous values $\in [-1, 1]$, for three items, positively correlated.
- scatterplot arranged along a 1-dimensional line $\rightarrow$ data matrix has a rank ~ 1 .



- $m \times n$ ratings matrix has a rank of $p \ll \min\{m, n\}$ the data can be approximately represented on a p -dimensional hyperplane.
- The missing ratings of a user can often be robustly estimated with as few as p specified entries as long as the p -dimensional hyperplane is known.

we can use a partially specified data set to recover the low-dimensional hyperplane on which the data approximately lies.

Intuition – *low rank*

- Assume all entries in R are observed
- Dimensionality reductions as matrix factorizations

$$R \in \mathbb{R}^{m \times n} \text{ of rank } k \ll \min\{m, n\}$$
$$R = U V^T$$
$$\begin{matrix} R^{m \times n} \\ U^{m \times k} \\ V^{n \times k} \end{matrix}$$

- **Columns of U :** basis vectors of the k -dimensional column space of R
- **j^{th} row of V :** contains the coefficients to combine these basis vectors into the j^{th} column of R .
- **Viceversa for columns of V .**

$$R^{m \times n} \text{ of rank } > k$$

$$R \approx U V^T$$

residual matrix $(R - UV)$

Example

R has correlations

		NERO	JULIUS CAESAR	CLEOPATRA	SLEEPLESS IN SEATTLE	PRETTY WOMAN	CASABLANCA
HISTORY	1	1	1	1	0	0	0
	2	1	1	1	0	0	0
	3	1	1	1	0	0	0
BOTH	4	1	1	1	1	1	1
	5	-1	-1	-1	1	1	1
ROMANCE	6	-1	-1	1	1	1	1
	7	-1	-1	-1	1	1	1

R

$\approx$

		HISTORY	ROMANCE
1	1	1	0
2	1	1	0
3	1	1	0
4	1	1	1
5	-1	1	1
6	-1	1	1
7	-1	1	1

U

$\times$

		NERO	JULIUS CAESAR	CLEOPATRA	SLEEPLESS IN SEATTLE	PRETTY WOMAN	CASABLANCA
HISTORY	1	1	1	1	0	0	0
ROMANCE	0	0	1	1	1	1	1

V^T

R can be approximately factorized into rank-2 factors

- $U \rightarrow$ basis for the column space. It shows the tastes of users for the 2 genres
- $V \rightarrow$ basis for the row space. It shows the membership of the movies to the 2 genres

- Columns of U and V as “latent concepts”
- Columns not always interpretable
- Columns not necessarily orthogonal

IMPORTANT: when R has missing values, one can still estimate all entries of the latent factors U and V, **because** data are inherently **redundant**. Then R can be estimated as $U V^T$

How do we predict (recommend) movies?

Matrix decomposition models are useful for very sparse datasets with potential **latent features**

Users (p)

	Movies (q)									
	1	0	0	0	2	0	3	0	4	0
	4	0	0	3	0	4	0	0	0	1
	0	0	4	0	0	0	1	0	3	0
	5	0	0	0	0	5	0	1	0	0
	0	0	4	0	3	0	2	0	0	0

Y

"sci-fi"

"horror"

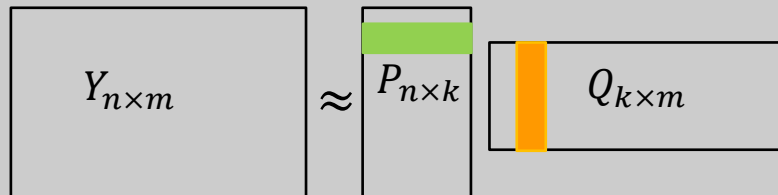
"romantic"

"comedy"

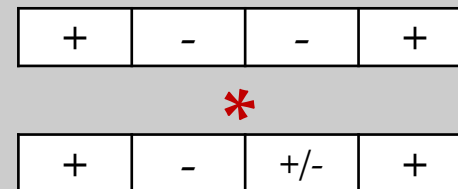
k latent features

$$Y_{i,j} \approx \mathbf{p}_i^T \cdot \mathbf{q}_j$$

$$Y_{n \times m} \approx P_{n \times k} \cdot Q_{k \times m}$$



Movies



Users

The Matrix Factorization idea

Rating matrix with missing values $R^{m \times n}$ approximately factorized in:

$$R \approx U V^T$$

- Each column of U (or V) is a latent vector
- Each row of U (or V) is referred to as a latent factor.
 - Row $\mathbf{u}_i$ of U is a user factor, contains k entries corresponding to the affinity of user i towards the k concepts in the ratings matrix.
 - Row $\mathbf{v}_i$ of V is an item factor, it represents the affinity of the i th item towards these k concepts.

Different matrix factorization methods impose different constraints on U and V and minimize different objective functions

Unconstrained Matrix Factorization

$$\underset{U, V}{\text{Minimize}} \ J = \frac{1}{2} \|R - UV^T\|^2$$

$\|\cdot\|^2$ is sum of the squares of the matrix entries (squared Frobenius norm)

Minimization happens only wrt the set S of observed entries in R .

$$S = \{(i, j) : r_{ij} \text{ is observed}\}$$

$$\underset{U, V}{\text{Minimize}} \ J = \frac{1}{2} \sum_{(i,j) \in S} e_{ij}^2 = \frac{1}{2} \sum_{(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2$$

1. Batch update (gradient descent)

$$\begin{aligned}\frac{\partial J}{\partial u_{iq}} &= \sum_{j:(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-v_{jq}) \quad \forall i \in \{1 \dots m\}, q \in \{1 \dots k\} \\ &= \sum_{j:(i,j) \in S} (e_{ij})(-v_{jq}) \quad \forall i \in \{1 \dots m\}, q \in \{1 \dots k\}\end{aligned}$$

$$\begin{aligned}\frac{\partial J}{\partial v_{jq}} &= \sum_{i:(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-u_{iq}) \quad \forall j \in \{1 \dots n\}, q \in \{1 \dots k\} \\ &= \sum_{i:(i,j) \in S} (e_{ij})(-u_{iq}) \quad \forall j \in \{1 \dots n\}, q \in \{1 \dots k\}\end{aligned}$$

$$U \Leftarrow U + \alpha EV$$

$$V \Leftarrow V + \alpha E^T U$$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N \{\mathbf{w}^T \phi(\mathbf{x}_n) - t_n\} \phi(\mathbf{x}_n)^T$$

where $E = R - UV^T$ and unobserved entries are set to 0

2. Stochastic Gradient Descent

$$u_{iq} \Leftarrow u_{iq} - \alpha \cdot \left[\frac{\partial J}{\partial u_{iq}} \right]_{\text{Portion contributed by } (i,j)} \quad \forall q \in \{1 \dots k\}$$

$$v_{jq} \Leftarrow v_{jq} - \alpha \cdot \left[\frac{\partial J}{\partial v_{jq}} \right]_{\text{Portion contributed by } (i,j)} \quad \forall q \in \{1 \dots k\}$$

$$u_{iq} \Leftarrow u_{iq} + \alpha \cdot e_{ij} \cdot v_{jq} \quad \forall q \in \{1 \dots k\}$$

$$v_{jq} \Leftarrow v_{jq} + \alpha \cdot e_{ij} \cdot u_{iq} \quad \forall q \in \{1 \dots k\}$$

$$\overline{u_i} \Leftarrow \overline{u_i} + \alpha e_{ij} \overline{v_j}$$

$$\overline{v_j} \Leftarrow \overline{v_j} + \alpha e_{ij} \overline{u_i}$$

- Advantages/disadvantages of batch vs SGD as usual...

... and with Regularization...

$$\text{Minimize}_{U, V} J = \frac{1}{2} \sum_{(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2 + \frac{\lambda}{2} \sum_{i=1}^m \sum_{s=1}^k u_{is}^2 + \frac{\lambda}{2} \sum_{j=1}^n \sum_{s=1}^k v_{js}^2$$

$$\begin{aligned} \frac{\partial J}{\partial u_{iq}} &= \sum_{j:(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-v_{jq}) + \lambda u_{iq} \quad \forall i \in \{1 \dots m\}, q \in \{1 \dots k\} \\ &= \sum_{j:(i,j) \in S} (e_{ij})(-v_{jq}) + \lambda u_{iq} \quad \forall i \in \{1 \dots m\}, q \in \{1 \dots k\} \end{aligned}$$

$$\begin{aligned} \frac{\partial J}{\partial v_{jq}} &= \sum_{i:(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right) (-u_{iq}) + \lambda v_{jq} \quad \forall j \in \{1 \dots n\}, q \in \{1 \dots k\} \\ &= \sum_{i:(i,j) \in S} (e_{ij})(-u_{iq}) + \lambda v_{jq} \quad \forall j \in \{1 \dots n\}, q \in \{1 \dots k\} \end{aligned}$$

where $E = R - UV^T$ and
unobserved entries are set to 0

$$U \leftarrow U(1 - \alpha \cdot \lambda) + \alpha EV$$

$$V \leftarrow V(1 - \alpha \cdot \lambda) + \alpha E^T U$$

batch

$$\overline{u_i} \leftarrow \overline{u_i} + \alpha(e_{ij}\overline{v_j} - \lambda\overline{u_i})$$

$$\overline{v_j} \leftarrow \overline{v_j} + \alpha(e_{ij}\overline{u_i} - \lambda\overline{v_j})$$

SGD

3. Alternating Least Square

$$\underset{U, V}{\text{Minimize}} \ J = \frac{1}{2} \|R - UV^T\|^2$$

- U and V are both unknown → J is not convex.
- If we fix one of the unknown, the optimization becomes quadratic and can be solved optimally!
- ALS techniques rotate between fixing the U and fixing V. When U is fixed, the system recomputes the V by solving a least-squares problem, and vice versa.

Adding biases

- μ global mean of the entire rating R
- o_i bias of user i
- p_j bias of item j

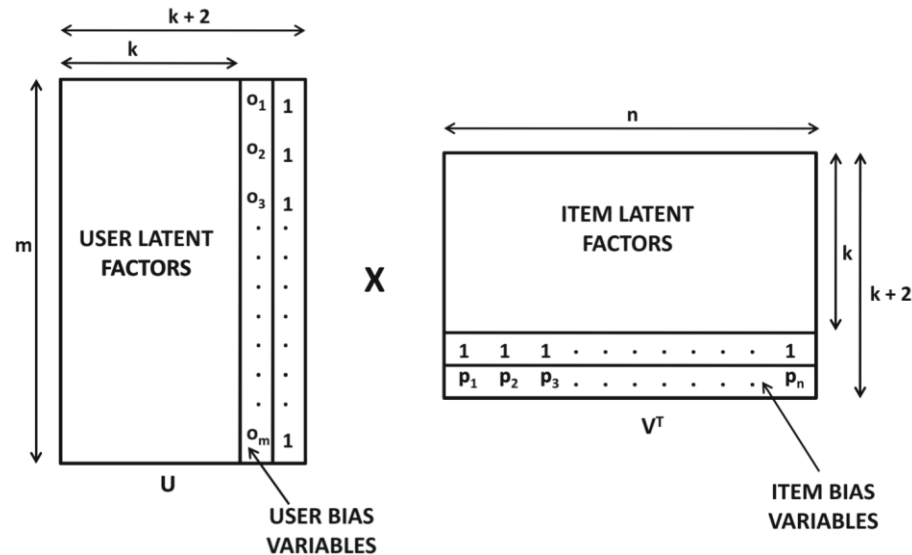
$$\widehat{r}_{ij} = \mu + o_i + p_j + \mathbf{u}_i \mathbf{v}_j^T$$

μ subtracted from R (pre-processing step)

$$\hat{r}_{ij} = o_i + p_j + \sum_{s=1}^k u_{is} \cdot v_{js}$$

$$\underset{\mathbf{U}, \mathbf{V}, \mathbf{o}, \mathbf{p}}{\text{Minimize } J} = \frac{1}{2} \sum_{(i,j) \in S} \left(r_{ij} - o_i - p_j - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2 + \frac{\lambda}{2} \left(\sum_{i=1}^m \sum_{s=1}^k u_{is}^2 + \sum_{j=1}^n \sum_{s=1}^k v_{js}^2 + \sum_{i=1}^m o_i^2 + \sum_{j=1}^n p_j^2 \right)$$

- we can increase the size of the factor matrices to incorporate these bias variables



$$\underset{U, V}{\text{Minimize}} J = \frac{1}{2} \sum_{(i,j) \in S} \left(r_{ij} - \sum_{s=1}^{k+2} u_{is} \cdot v_{js} \right)^2 + \frac{\lambda}{2} \sum_{s=1}^{k+2} \left(\sum_{i=1}^m u_{is}^2 + \sum_{j=1}^n v_{js}^2 \right)$$

subject to:

$(k+2)$ th column of U contains only 1s

$(k+1)$ th column of V contains only 1s

*"1s" are restored
after each update*

Singular Value Decomposition

- **Truncated SVD** of rank $k \ll \min\{m, n\}$

$$R \approx Q_k \Sigma_k P_k^T \quad Q_k^{m \times k}, \Sigma_k^{k \times k}, P_k^{n \times k}$$

$$U = Q_k \Sigma_k$$

$$V = P_k$$

$$\text{Minimize } J = \frac{1}{2} \|R - UV^T\|^2$$

subject to:

Columns of U are mutually orthogonal

Columns of V are mutually orthogonal

But our R has missing values...

A Simple Iterative Approach using SVD on R with missing values.

1. Initialize the missing entries in the i^{th} row of R to be the mean of row i , call that matrix R_f
2. Perform rank- k SVD of R_f to obtain: $Q_k \Sigma_k P_k^T$
3. Readjust only the (originally) missing entries of R_f to the corresponding values in $Q_k \Sigma_k P_k^T$. GOTO 1.

The final ratings matrix is then given by $Q_k \Sigma_k P_k^T$ at convergence.

Regularization can be used (regularized SVD).

Look at the example on page 117 of the book.

.....

Learning the parts of objects by non-negative matrix factorization

[*Nature*, 1999]

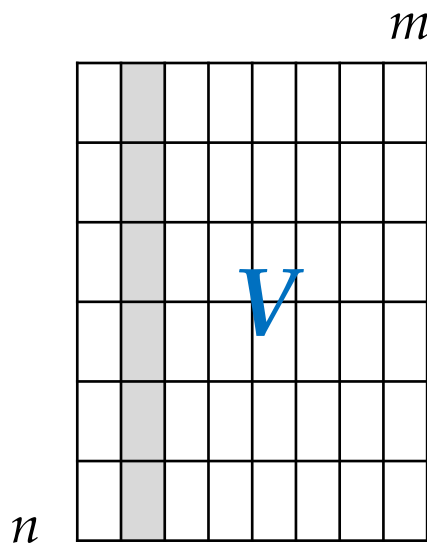
Daniel D. Lee^{*} & H. Sebastian Seung^{*†}

^{} Bell Laboratories, Lucent Technologies, Murray Hill, New Jersey 07974, USA*

[†] Department of Brain and Cognitive Sciences, Massachusetts Institute of Technology, Cambridge, Massachusetts 02139, USA

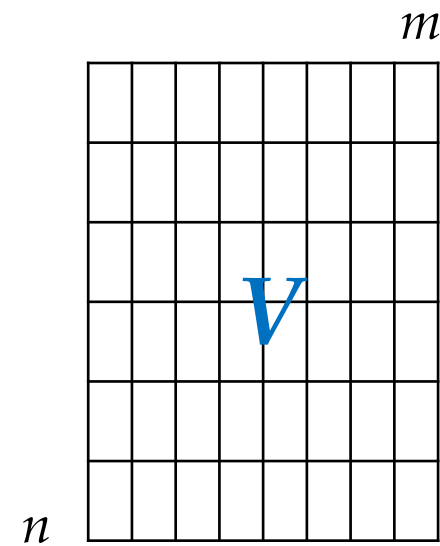
This paper is
a mandatory
reading

- algorithm that is able to learn parts of faces and semantic features of text.
- **non-negativity constraints:** lead to a parts-based representation because they allow only additive, not subtractive, combinations.



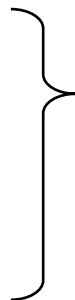
m images of faces
(one face per column)
each face has n dimension

V_{ij} non-negative



m pieces of text
n words
 V_{ij} = number of occurrences of
word i in text j

1. Vector quantization
2. PCA
3. NMF

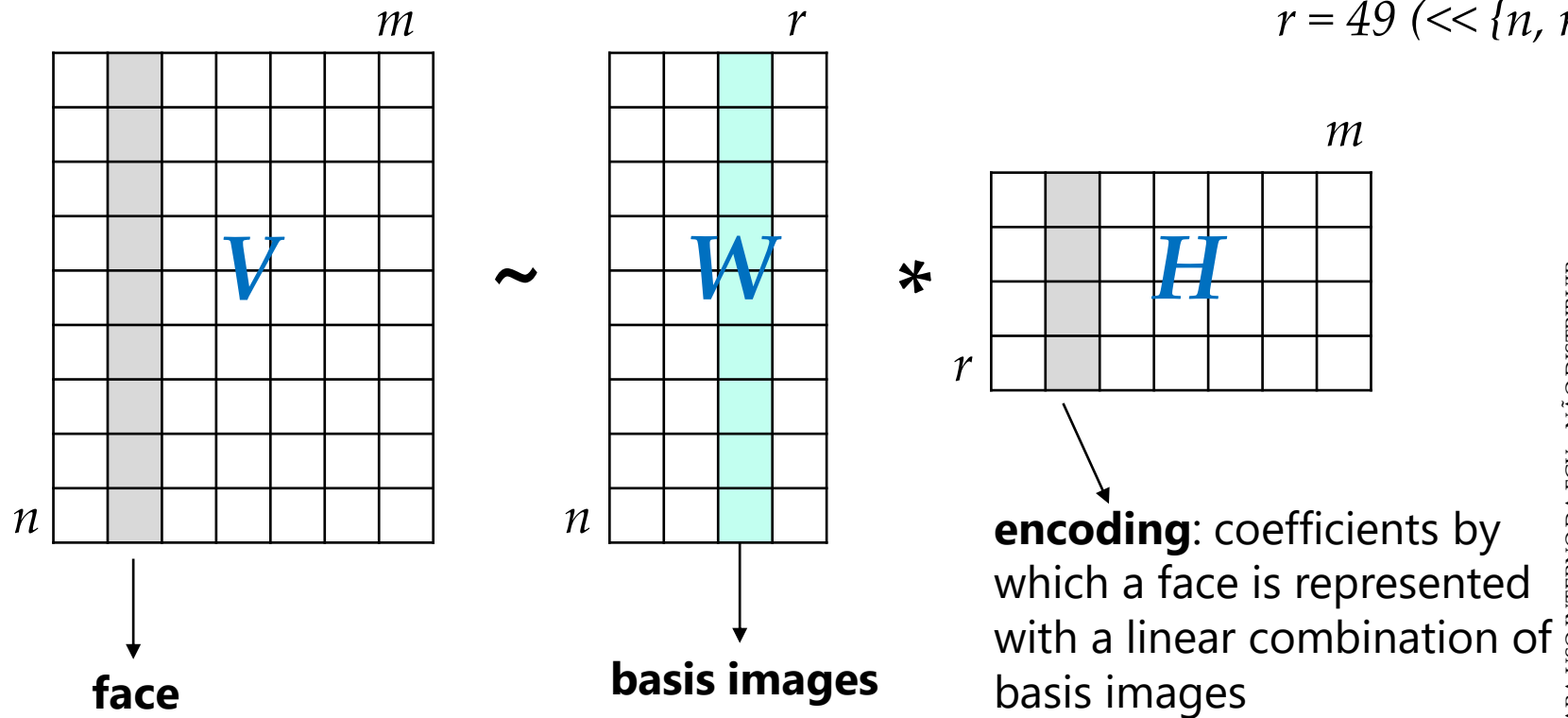


$$V \approx WH$$

$$V_{i\mu} \approx (WH)_{i\mu} = \sum_{a=1}^r W_{ia} H_{a\mu}$$

The FACES problem

$$\begin{aligned}m &= 2429 \\n &= 19 \times 19 = 361 \\r &= 49 \ll \{n, m\}\end{aligned}$$



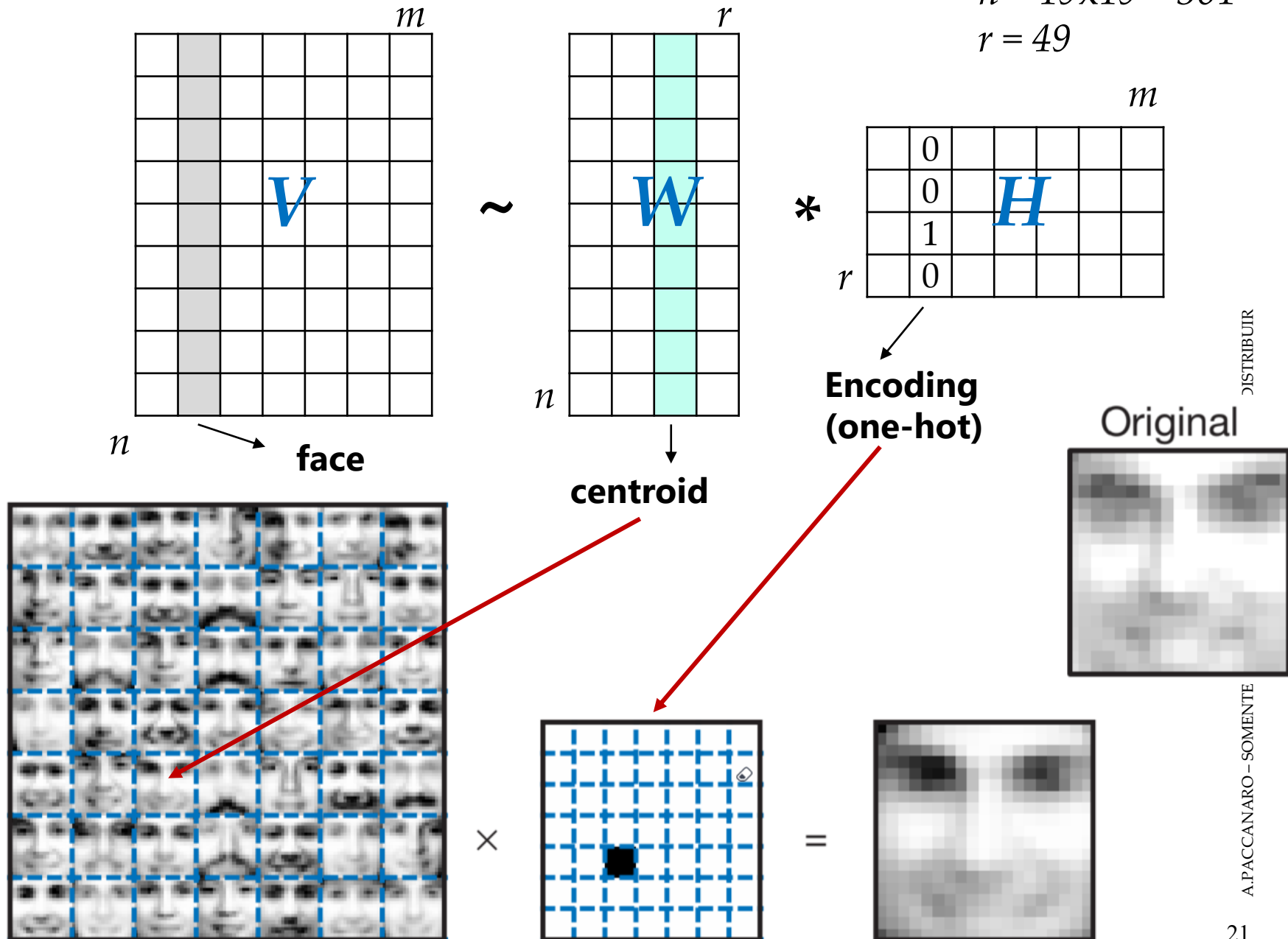
The differences between PCA, VQ and NMF arise from different constraints imposed on the matrix factors W and H .

1. Vector quantization (~ k-means)

$m = 2429$

$n = 19 \times 19 = 361$

$r = 49$



Constraint on the decomposition: Each column of H is constrained to be a unary vector, with one element equal to unity and the other elements equal to zero.

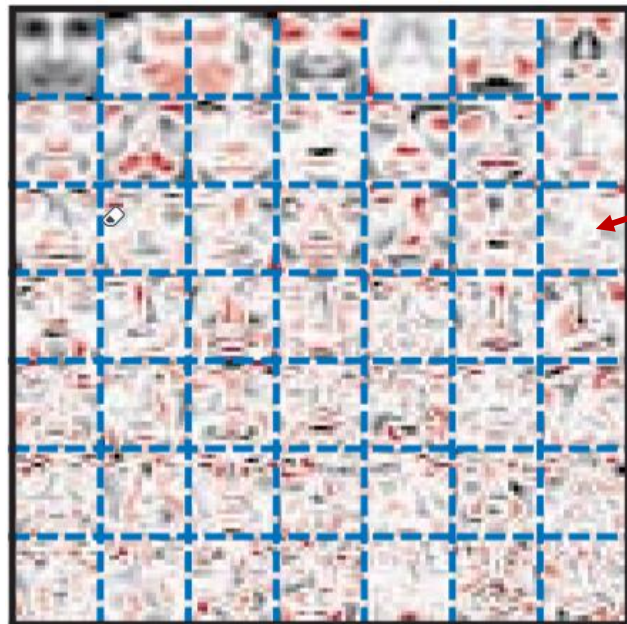
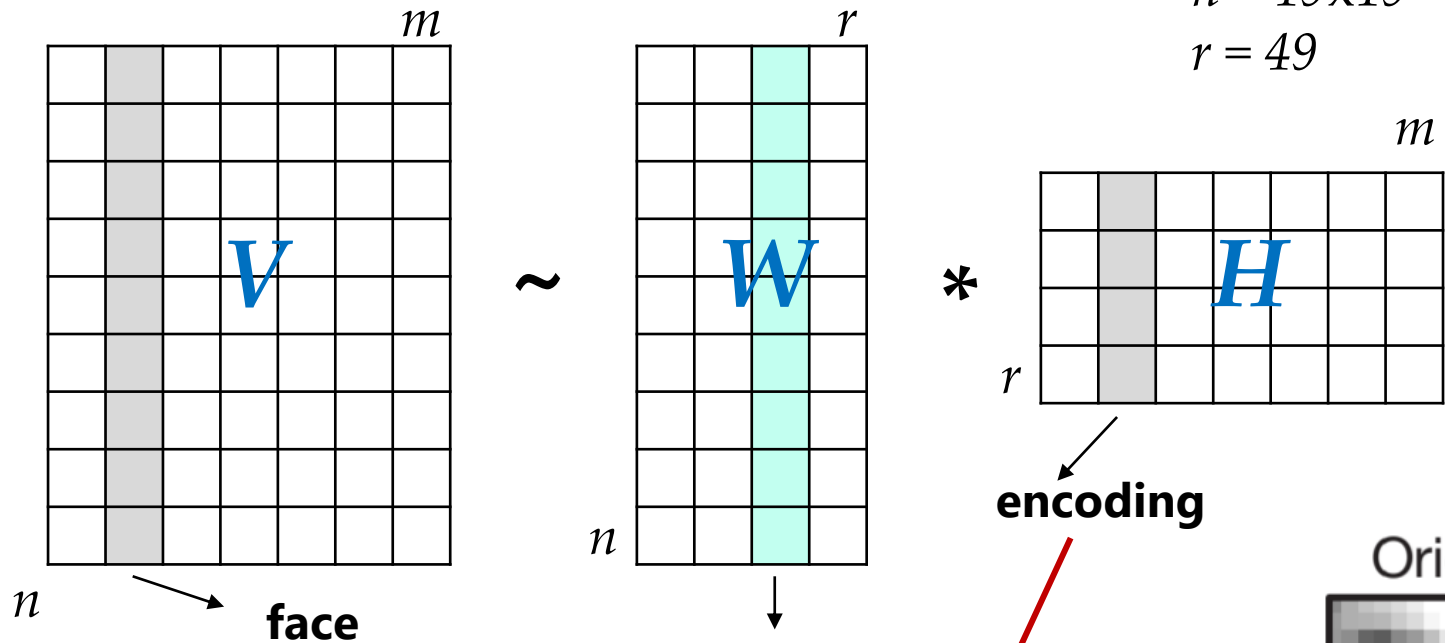
- Every face is approximated by a single basis image in the factorization $V \sim WH$.
- Therefore, it **learns basis images that are prototypical faces**

2. Principal Component Analysis

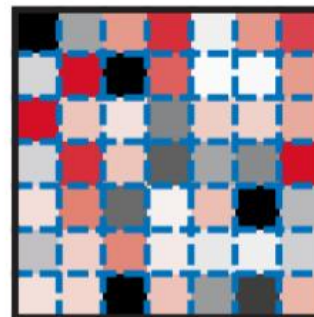
$$m = 2429$$

$$n = 19 \times 19 = 361$$

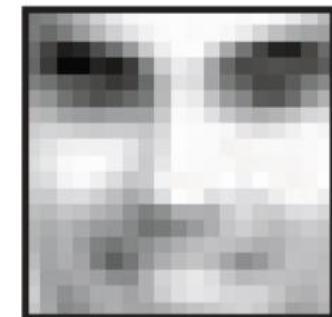
$$r = 49$$



$\times$



$=$



Original



ISTRIBUIR

A.PACCANARO - SOMENTE I

Constraint on the decomposition: PCA constrains the columns of W to be orthonormal to each other.

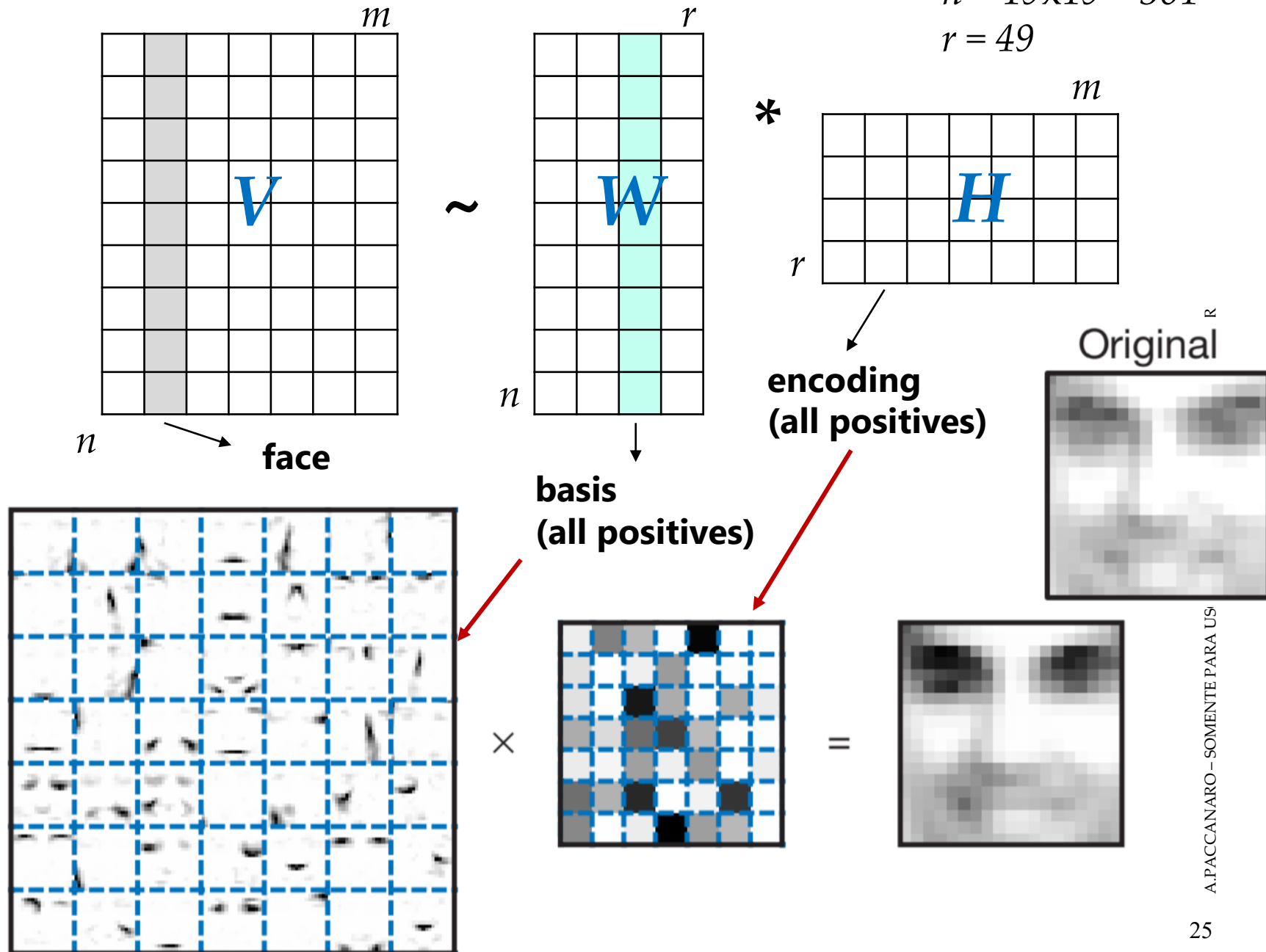
- This allows a distributed representation in which each face is approximated by a linear combination of **all** the basis images
- eigenvectors do not have a visual interpretation because they are used in linear combinations that involve cancellations between positive and negative numbers.

1. NMF

$$m = 2429$$

$$n = 19 \times 19 = 361$$

$$r = 49$$



Constraint on the decomposition: W and H have non-negative entries.

- allows the combination of multiple basis images to represent a face (like PCA, but not VQ).
- only additive combinations are allowed, because the non-zero elements of W and H are all positive → no subtractions can occur.
- *The non-negativity constraints are compatible with the intuitive notion of combining parts to form a whole*

→ NMF learns a parts-based representation.

- **basis images are sparse** because they are non-global and contain several versions of mouths, etc
- **encoding are sparse** because a face does not use all the available parts.

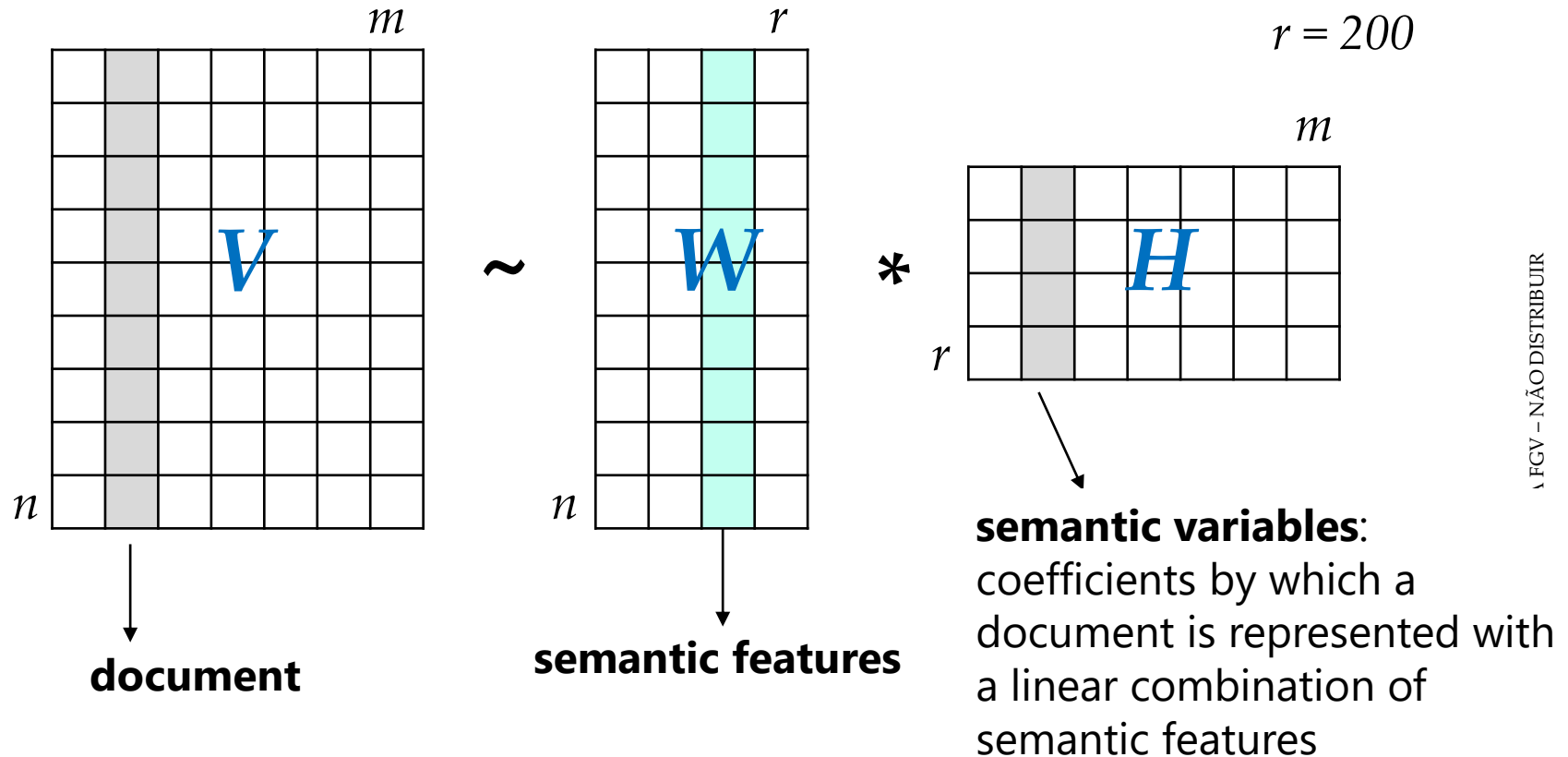
distributed representation in which each face is approximated by a linear combination of few basis

The semantic analysis of text problem

$$m = 30991$$

$$n = 15276$$

$$r = 200$$



The differences between PCA, VQ and NMF arise from different constraints imposed on the matrix factors W and H .

1. Vector quantization (~ k-means)

- A single centroid is active for each document.
- If the same centroid is active for a group of documents, they are semantically related
- Elements of H are called *semantic variables*, and they are used for automatic semantic indexing of documents by topic.
- Each column of W , or *semantic feature*, consists of the word frequencies for the corresponding semantic variable.
- Only one semantic variable per document, which prevents more than one topic from being attributed to a document.

2. Principal Components Analysis

- Semantic variables are difficult to interpret because:
 - all semantic variables are used to represent each document
 - and negative values for semantic variables are allowed.
- It would make more sense for each document to be associated with some small subset of a large array of topics, rather than just one topic (like VQ) or all the topics (like PCA).

3. NMF

court government council culture supreme constitutional rights justice	president served governor secretary senate congress presidential elected
flowers leaves plant perennial flower plants growing annual	disease behaviour glands contact symptoms skin pain infection

×



≈

Encyclopedia entry:
'Constitution of the
United States'

president (148)
congress (124)
power (120)
united (104)
constitution (81)
amendment (71)
government (57)
law (49)

*distributed representation in
which each document is
approximated by a linear
combination of few semantic
variables*

metal process method paper ... glass copper lead steel
person example time people ... rules lead leads law

- In each semantic feature, the algorithm has grouped together semantically related words.
- The algorithm uses context to differentiate between multiple meanings of the same word

Learning NMF (we go back to our textbook)

$$\text{Minimize } J = \frac{1}{2} ||R - UV^T||^2$$

subject to:

$$\begin{cases} U \geq 0 \\ V \geq 0 \end{cases}$$

Multiplicative update rule (proof in Lee & Seung, NIPS 2000 paper)

$$u_{ij} \Leftarrow \frac{(RV)_{ij} u_{ij}}{(UV^T V)_{ij} + \epsilon} \quad \forall i \in \{1 \dots m\}, \forall j \in \{1 \dots k\}$$
$$v_{ij} \Leftarrow \frac{(R^T U)_{ij} v_{ij}}{(V U^T U)_{ij} + \epsilon} \quad \forall i \in \{1 \dots n\}, \forall j \in \{1 \dots k\}$$

- $\epsilon = 10^{-9}$ used to increase numerical stability
- All entries in U and V updated simultaneously
- U and V initialized at small random values, iterate until convergence

Regularized NMF (Pauca et al, Lin. Alg. & Appl., 2005)

$$F(W, H) = \frac{1}{2} \|Y - WH\|_F^2 + \frac{\alpha}{2} \|W\|_F^2 + \frac{\beta}{2} \|H\|_F^2$$

Algorithm 2 (CNMF). Calculate W and H such that $Y \approx WH$

Require: $m \times n$ matrix $Y \geq 0$, and $k > 0, k \ll \min(m, n)$

$W_{ij} \Leftarrow$ nonnegative value ($1 \leq i \leq m, 1 \leq j \leq k$)

$H_{ij} \Leftarrow$ nonnegative value ($1 \leq i \leq k, 1 \leq j \leq n$)

Scale columns of W to sum to one.

while converge or stop **do**

$$H_{cj}^{(t)} \leftarrow H_{cj}^{(t-1)} \frac{((W^{(t-1)})^T Y)_{cj} - \beta H_{cj}^{(t-1)}}{((W^{(t-1)})^T W^{(t-1)} H^{(t-1)})_{cj} + \epsilon} \quad (1 \leq i \leq m, 1 \leq c \leq k)$$

$$W_{ic}^{(t)} \leftarrow W_{ic}^{(t-1)} \frac{(Y(H^{(t)})^T)_{ic} - \alpha W_{ic}^{(t-1)}}{(W^{(t-1)} H^{(t)} (H^{(t)})^T)_{ic} + \epsilon} \quad (1 \leq c \leq k, 1 \leq j \leq n)$$

Scale columns of W to sum to one.

end while

Interpretability example (from the textbook)

		MILK	YOGURT	CHEESE	JUICE	SODA	BEER
DAIRY	1	2	2	1	2	0	0
	2	2	3	3	3	0	0
	3	1	1	1	1	0	0
BOTH	4	2	2	2	3	1	1
	5	0	0	0	1	1	1
DRINKS	6	0	0	0	2	1	2

R

$\approx$

		DAIRY	DRINKS
1	2	0	
2	3	0	
3	1	0	
4	2	1	
5	0	1	
6	0	2	

U

$\times$

		MILK	YOGURT	CHEESE	JUICE	SODA	BEER
DAIRY	1	1	1	1	1	0	0
DRINKS	0	0	0	1	1	1	1

V^T

$$r_{ij} \approx (\text{Affinity of user } i \text{ to dairy aspect}) \times (\text{Affinity of item } j \text{ to dairy aspect}) \\ + (\text{Affinity of user } i \text{ to drinks aspect}) \times (\text{Affinity of item } j \text{ to drinks aspect})$$